

PROYECTO FINAL

Descripción:

- Cada grupo de estudiantes elegirá un proyecto de los listados a continuación, podrán hacer avances en el problema cada semana, y lo entregarán en el repositorio de GitHub **con un resumen corto** (archivo PDF) de la implementación en la última semana del curso.
- Estos proyectos serán la aplicación de lo aprendido en clases, si se desea plantear un proyecto diferente a los planteados, la proposición deberá estar en el marco de lo visto en clases.
- La colaboración en equipo es importante, un resumen de participación de cada miembro deberá estar incluida en el informe final.

Se evaluará los siguientes aspectos:

- Facilidad de explicar su propuesta en un párrafo de **no más de 250 palabras**
- Manejo de terminología técnica asociada a los temas vistos en clase en el resumen corto de implementación
- Criticidad: Se espera un apartado que explique las dificultades del problema y como se asocian a lo visto en clase
- Implementación funcional, el código implementado debe correr y ser ejecutable
- Conclusiones del Proyecto: las respuestas obtenidas se aproximan a lo esperado
- Colaboración en equipo: Un apartado que explica la colaboración de cada integrante
- Exposición corta del proyecto, tiempo **no mayor a 15 minutos**, sobrepasar este tiempo **será penalizado**
- Más **detalles de la evaluación del proyecto final** se encuentra en la página inicial del curso.

1. VEHICLE ROUTING PROBLEM VRP

Contexto: Una empresa de entregas dispone de una flota de vehículos con capacidad limitada, y debe repartir paquetes a un conjunto de clientes ubicados en distintas direcciones. Todos los vehículos parten desde un único depósito y deben regresar al mismo al finalizar su ruta.

Cada cliente debe ser visitado exactamente una vez por un solo vehículo, y la cantidad de paquetes que transporta un vehículo no debe superar su capacidad.

Objetivo: Diseñar las rutas para todos los vehículos de manera que la distancia total recorrida sea mínima, cumpliendo con todas las restricciones de capacidad y asignación.

Se plantea un dataset en la carpeta VRP/data, resolver los problemas de acuerdo con:

- a) Small-vrp: contiene la ubicación de los clientes y sus demandas, resuelve el problema con una flota de 2, 3 y 4 vehículos con sus respectivas capacidades
- b) Medium-vrp: Cuál sería la flota necesaria para obtener las rutas más optimas si se asume:
 - 1.1. Los vehículos tienen capacidad de 100
 - 1.2. ¿Qué capacidad deberían tener los vehículos si se desea cubrir todo con 4 rutas y cuáles serían las rutas optimas?
 - 1.3. ¿Si los 4 vehículos tienen capacidad máxima de 150, cuando el problema se vuelve inviable? ¿Cuál sería la flota ideal para este problema?
- c) Large-vrp: Aplicar cualquier metaheurística o algoritmo que permita resolver el problema en tiempo razonable:
 - 1.1. ¿Sería adecuado una flota de 10 vehículos?
 - 1.2. Subir o bajar el número de vehículos beneficia o perjudica a las rutas
 - 1.3. ¿Cuáles serían las rutas y la flota óptima para este problema?

2. PACKING PROBLEM (OPTIMIZACIÓN DE ESPACIO EN UNA CAJA)

Contexto: Ubicar objetos dentro de un contenedor sin que se superpongan y ocupando el menor espacio posible, es decir reducir el espacio ocupado colocando tantos objetos como sea posible sin dejar huecos. Cada individuo es una configuración distinta de cómo se ubican los objetos, y la función de fitness minimizará el espacio vacío y evitará solapamiento. Se recomienda el uso de algoritmos metaheurísticos para su resolución.

Objetivo: Encontrar la configuración adecuada, de manera que se ocupe el espacio lo óptimo posible, es decir sin huecos.

Se plantea un dataset (*10Figures.json*, *25Figures.json*, *50Figures.json*) en la carpeta PackingProblem/data, resolver cada problema de acuerdo con:

- a) Sin rotación de figuras. ¿Cuál es el área mínima en el que caben todas las figuras?
- b) ¿En un área de 50x50, sin usar prioridad cuales serían las figuras que maximizan el espacio ocupado?
- c) ¿En un área de 50x50, con prioridad cuales serían las figuras que maximizan el espacio ocupado?
- d) Bonus: Implementación con rotación de figuras

3. CREACIÓN DE UN AGENTE RAG (RETRIEVAL-AUGMENTED GENERATION)

Contexto: crear un asistente personalizado que pueda operar de forma local, accediendo a información específica contenida en datasets propios de una empresa, organización o individuo.

Objetivo: Desarrollar un agente virtual inteligente capaz de responder preguntas en lenguaje natural utilizando información contenida en un dataset local, combinando técnicas de búsqueda informada y no informada junto a modelos LLM locales (vía Ollama)

- a) La implementación es un prototipo simple funcional
- b) Se puede usar algoritmos de búsqueda implementados localmente en combinación con prompts hacia el agente
- c) El agente deberá responder en concordancia con el data set entregado, es decir se le podrá preguntar cosas específicas y su respuesta será la que se encuentra en el dataset.
- d) ¿Como pudiera el agente guardar los archivos que se le ha proporcionado, que estructura seria la más adecuada?
- e) Debería ser capaz de responder a preguntas como:
 - ¿Cuál es el orden cronológico de los documentos?
 - ¿Cómo se llama la empresa y cuál es el perjuicio ocasionado?
 - ¿Qué tipo de problema fue, cuál fue la causa del incidente?
 - ¿Quiénes son los involucrados, se conoce el nombre de quien reporta el incidente?
 - ¿Qué archivo describe las sanciones ocasionadas?
 - ¿Cuáles son los archivos que el agente tiene disponible para proporcionar información?
- f) Investigar como evaluar un agente RAG, evaluar el agente al menos con un tipo de métrica

4. RESOLUCIÓN DEL TSP MEDIANTE ALGORITMOS GENÉTICOS

Contexto: Resolver el TSP usando algoritmos metaheurísticos, en este caso el uso de Algoritmos Genéticos. El tamaño de los TSP a resolver deberán superar las 100 ciudades como mínimo.

Objetivo: Diseñar, implementar y analizar un Algoritmo Genético (AG) para resolver instancias del TSP, evaluando tanto la calidad de las soluciones como el comportamiento del algoritmo.

- a) Representar cada individuo como una permutación válida de ciudades. Garantizar que todas las soluciones generadas (inicialización, crossover, mutación) sean válidas (justificar la representación utilizada):
 - Sin ciudades repetidas

- Sin ciudades faltantes
- b) Definición de operadores genéticos (justificar el uso):
- Un operador de **crossover válido para permutaciones** (ej: PMX, OX o CX)
 - Un operador de **mutación** (ej: swap, inversión, scramble)
- c) Realizar análisis de la evolución del fitness:
- Graficar por generación:
 - Mejor fitness
 - Fitness promedio
 - Peor fitness
 - Convergencia del algoritmo
 - Análisis de presencia de estancamiento o convergencia prematura
- d) Medición y análisis de diversidad poblacional:
- Definir una métrica de diversidad: Distancia promedio entre individuos y/o porcentaje de individuos únicos
 - Graficar la diversidad a lo largo de las generaciones, analizar la relación entre:
 - Diversidad y calidad de solución
 - Diversidad y convergencia
- e) Evaluación de la calidad de las soluciones
- Comparar los resultados del AG con al menos: Una heurística simple (ej: nearest neighbor) o implementación previa
 - Reportar fitness final y Error relativo (cuando sea posible)
- f) Análisis experimental de parámetros
- Reportar resultados en tablas
 - Análisis de: Tamaño de población, Parametrización del crossover, Probabilidad de mutación